Test Cases and Maintenance Plan Summary: Health Management System (HMS)

Aim

This document delineates the comprehensive quality assurance strategies, critical test cases, and the post-deployment maintenance architecture for the Health Management System (HMS). The objective is to validate system reliability, ensure stringent data security (e.g., HIPAA compliance), and establish a sustainable operational lifecycle through proactive maintenance protocols.

Core System Test Cases

The testing strategy employs a multi-tiered approach validating workflow, security, and interoperability. Below is a subset of the critical test scenarios implemented:

Test ID	Module	Expected Result
TC-001	Authentication	System grants access and redirects to role-specific dashboard. Account temporarily locked; alert sent to security admin.
TC-002	Patient Portal	Appointment confirmed, calendar updated, and confirmation dispatched.
TC-003	Provider Portal	Patient history, lab results, and past notes render correctly and securely. Prescription is securely transmitted to the integrated pharmacy API without errors.
TC-004	Admin Portal	Access denied; RBAC enforces strict separation of duties.

TC-005	Security	System neutralizes input, returns zero database errors, and logs malicious attempt.

Post-Deployment Maintenance Plan

To ensure the HMS remains highly available, secure, and compliant with evolving healthcare regulations, maintenance is categorized into four paradigms:

- **Preventive Maintenance**
  - **Patch Management:** Weekly application of security patches and server OS updates.
  - **Data Backup:** Daily differential and weekly full encrypted backups with georedundant storage and DR drills.
- **Corrective Maintenance**
  - **Hotfixes & SLAs:** Rapid resolution of software bugs or broken integrations. SLAs mandate a response time of <1 hour for critical system downtime.
- **Adaptive Maintenance**
  - **Regulatory Compliance:** Iterative updates to system logic conforming to new privacy laws (HIPAA/GDPR) or medical coding standards (ICD-10).
  - **API Updates:** Maintaining compatibility with third-party billing, pharmacy, and laboratory systems.
- **Perfective Maintenance**
  - **Performance Tuning:** Ongoing database query optimization to decrease load times for complex EHR datasets.
  - **UX Enhancements:** Streamlining clinical workflows based on user feedback to reduce medical practitioner click-fatigue.

Conclusion

Developing a robust Health Management System requires balancing technological innovation with strict regulatory compliance. Agile and DevOps methodologies provide the ideal framework for secure, iterative deployment. By combining rigorous project governance, comprehensive testing, and proactive maintenance, organizations ensure a resilient, efficient, and compliant platform that ultimately enhances patient care delivery.